



PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL FORMATO GUÍA DE APRENDIZAJE

1) IDENTIFICACIÓN DE LA GUIA DE APRENDIZAJE

- **Denominación del Programa de Formación:** tecnología en Análisis y Desarrollo de Software
- **Código del Programa de Formación:** 3234206
- **Nombre del Proyecto:** Construcción de software integrador de tecnologías orientadas a servicios
- **Fase del Proyecto:** Ejecución
- **Actividad de Proyecto:** Introducción a los operadores modernos
- **Competencia:** DESARROLLAR LA SOLUCIÓN DE SOFTWARE DE ACUERDO CON EL DISEÑO Y METODOLOGÍAS DE DESARROLLO
- **Resultados de Aprendizaje Alcanzar:** Resolver procesos lógicos a través de la implementación de Algoritmos y el lenguaje de programación seleccionado.
- **Duración de la Guía:** 6 Horas 15 minutos

2) PRESENTACIÓN

Aprendiz SENA:

Después de recorrer juntos los conceptos fundamentales de la programación —variables, constantes, arreglos, operadores lógicos, condicionales y ciclos— hemos alcanzado un punto clave en tu proceso de formación. Lo aprendido hasta ahora no solo te ha permitido comprender cómo piensa un programa, sino también cómo organizar soluciones paso a paso. Ese dominio inicial es el cimiento sobre el cual construiremos nuevas habilidades que te acercarán aún más al pensamiento lógico propio de un desarrollador profesional.

En esta guía damos un paso hacia un JavaScript más moderno, más expresivo y cercano a las prácticas actuales del sector. Exploraremos herramientas que te permitirán escribir código más ordenado, más flexible y eficiente: **parámetros rest, operador spread, destructuración y manejo de errores**. Estos conceptos no son simples añadidos; son



recursos que transformarán la forma en que interactúas con los datos, estructuras tus funciones y respondes ante situaciones inesperadas en tus programas.

Cada tema está diseñado para retarte, invitarte a experimentar y llevarte a descubrir nuevas maneras de resolver problemas. Esta guía es una oportunidad para avanzar con seguridad, fortalecer tu lógica y prepararte para los desafíos del desarrollo de software real. Si te apasiona crear, mejorar y entender cómo funciona lo que construyes, aquí encontrarás un camino estimulante para seguir creciendo.

3) FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

A. Actividades de reflexión inicial.

Un equipo de desarrollo está construyendo una pequeña aplicación que procesa listas de datos enviadas por los usuarios. Al intentar ampliar la funcionalidad, se dan cuenta de que:

- El código se vuelve difícil de leer cuando se manejan muchos parámetros en una función.
- Reutilizar partes de un arreglo o un objeto resulta más complejo de lo esperado.
- Pequeños errores en los datos de entrada causan fallas que detienen toda la aplicación.
- Los programadores sienten que están repitiendo código en varias partes del proyecto.

Preguntas para reflexionar (sin buscar solución técnica):

1. ¿Qué dificultades podrían enfrentar los desarrolladores al trabajar con datos más extensos o estructuras más complejas?
2. ¿Cómo impacta en la calidad del software el no manejar adecuadamente los errores?
3. ¿Por qué un código desordenado o poco flexible puede afectar la colaboración dentro de un equipo?
4. ¿Qué importancia tiene contar con herramientas que permitan manipular datos de una forma más clara y organizada?

El aprendiz debe analizar la situación y responder desde su razonamiento lógico y experiencia previa, sin usar aún conceptos técnicos avanzados.

Estrategias didácticas activas:



- Analogía con la vida diaria
- Lluvia de ideas
- Discusión guiada
- Aprendizaje basado en problemas (ABP)

Duración de la actividad 15 minutos.

B. Actividades de contextualización e identificación de conocimientos necesarios para el aprendizaje.

En la reflexión inicial identificamos que, a medida que un proyecto crece, la manipulación de datos y la organización del código pueden convertirse en retos significativos. Ahora, en este momento de contextualización, comenzaremos a comprender cómo JavaScript ofrece herramientas modernas que permiten enfrentar estos problemas de forma eficiente y profesional.

1. Parámetros REST

En muchos programas, especialmente cuando trabajamos con funciones, no siempre sabemos cuántos valores vamos a recibir.

Por ejemplo:

- Una función puede necesitar procesar **una lista variable de datos**.
- En algunas situaciones queremos **sumar, filtrar o analizar** un número distinto de argumentos cada vez.
- Si intentamos declarar un parámetro para cada posible dato, el código se vuelve inflexible y difícil de mantener.
- Además, JavaScript solo permite capturar argumentos adicionales a través del objeto arguments, que no es un arreglo real y complica el trabajo.

Esto genera funciones rígidas, difíciles de adaptar y propensas a errores cuando los valores cambian.

Los parámetros rest aparecen como una solución moderna y clara a este problema.



Los **parámetros rest** son una característica de JavaScript que permite que una función reciba **un número indefinido de argumentos**, agrupándolos automáticamente dentro de un **arreglo**.

Se representan con tres puntos (...) antes del nombre del parámetro.

```
function listarElementos(...valores) {  
  for (let i = 0; i < valores.length; i++) {  
    console.log("Elemento", i + 1, ":", valores[i]);  
  }  
}
```

```
listarElementos("HTML", "CSS", "JavaScript", "Git");
```

2. Problema de Mutabilidad

Cuando trabajamos con objetos o arreglos en JavaScript, si creamos una “copia” utilizando asignación directa, en realidad no estamos copiando el valor, sino la referencia. Esto significa que **si modificamos la copia, también se modifica el original**, incluso si no es lo que queremos.

```
const usuarioOriginal = {  
  nombre: "Carlos",  
  edad: 25  
};
```

// Copia incorrecta: ambas variables apuntan al mismo objeto

```
const usuarioCopia = usuarioOriginal;
```

// Modificamos la "copia"

```
usuarioCopia.edad = 30;
```



```
console.log(usuarioOriginal.edad); // 30 Error: el original también cambió  
console.log(usuarioCopia.edad); // 30
```

¿Qué pasó aquí?

- usuarioCopia **no es una copia real**.
- Ambas variables apuntan al **mismo objeto en memoria**.
- Al cambiar la copia, cambiamos involuntariamente el original.
- Esto causa errores muy difíciles de detectar en programas reales.

Solución: Operador Spread + Inmutabilidad

El operador spread permite crear una copia independiente, lo que significa que:

- Podemos modificar la copia sin afectar el original.
- Conservamos el estado previo.
- El código es más seguro y predecible.

Solución

```
const usuarioOriginal = {  
  nombre: "Carlos",  
  edad: 25  
};
```

```
// Copia correcta: se crea un nuevo objeto  
const usuarioCopia = { ...usuarioOriginal };
```

```
// Modificamos la copia  
usuarioCopia.edad = 30;
```



```
console.log(usuarioOriginal.edad); // 25 El original NO se modifica
```

```
console.log(usuarioCopia.edad); // 30
```

¿Qué resolvemos aquí?

- Los cambios en la copia no afectan al original.
- Podemos mantener versiones anteriores del estado.
- El código se vuelve más estable y fácil de depurar.
- Se aplica el principio de inmutabilidad de forma sencilla.

3. Operador Spread

El operador Spread permite copiar o expandir estructuras sin afectar los valores originales. Esto evita errores derivados de la mutación directa y facilita un código más seguro y predecible. Se ubica ... antes del nombre del arreglo u objeto para copiar su contenido.

En JavaScript, trabajar con arreglos y objetos puede volverse complejo cuando necesitamos:

- **Copiar** un arreglo u objeto sin afectar el original.
- **Combinar** varias estructuras de datos.
- **Agregar** nuevos elementos sin modificar directamente los existentes.
- **Expandir** los elementos de un arreglo u objeto dentro de otro.

Si intentamos hacerlo de manera tradicional:

- Se generan **copias por referencia**, lo que puede causar errores difíciles de detectar.
- Una modificación no intencional a la copia afecta el valor original.
- El código se vuelve más largo y propenso a fallos.

Es decir, cuando queremos manipular datos sin romper su integridad, los métodos manuales son incómodos y riesgosos.

El operador spread surge precisamente para resolver estas situaciones de una forma limpia y segura.



El **operador spread** (...) permite **expandir** los elementos de un arreglo u objeto dentro de otra estructura, creando copias independientes o combinando datos de forma sencilla.

Se utiliza para:

- Copiar arreglos y objetos sin modificar los originales.
- Combinar estructuras de datos.
- Pasar varios valores como si fueran argumentos individuales.

Ejemplo:

```
const numeros = [1, 2, 3];  
const copia = [...numeros];
```

```
console.log(copia); // [1, 2, 3]
```

Aquí copia es una nueva versión del arreglo, independiente del original.

4. Destructuración

Cuando trabajamos con objetos o arreglos en JavaScript, es común que necesitemos acceder a varios de sus valores.

Sin embargo, hacerlo de forma tradicional puede generar:

- **Código repetitivo**, accediendo a cada valor una y otra vez.
- **Falta de claridad**, especialmente cuando los valores tienen nombres largos.
- **Mayor probabilidad de errores**, por escribir propiedades o índices incorrectos.
- **Baja legibilidad**, dificultando la comprensión del propósito del código.

```
const persona = {  
  nombre: "Laura",  
  edad: 28,  
  ciudad: "Bogotá"  
};
```

// Forma tradicional: repetitiva e incómoda

```
const nombre = persona.nombre;
```



```
const edad = persona.edad;  
const ciudad = persona.ciudad;  
  
console.log(nombre, edad, ciudad);
```

Aquí el código es funcional, pero:

- Repetimos el nombre del objeto varias veces.
- No es tan claro visualmente.
- En objetos más grandes, se vuelve tedioso.

La **destructuración** es una característica de JavaScript que permite **extraer valores** de arreglos y objetos de manera más clara, directa y organizada, utilizando una sintaxis compacta.

En lugar de acceder valor por valor, la destructuración **toma los elementos que necesitamos y los asigna a variables automáticamente**.

Ejemplo:

```
const persona = {  
  nombre: "Laura",  
  edad: 28,  
  ciudad: "Bogotá"  
};  
  
// Destructuración: clara y limpia  
const { nombre, edad, ciudad } = persona;  
  
console.log(nombre, edad, ciudad);
```

¿Qué resolvemos?

- Reducimos código repetitivo.



- Evitamos errores al acceder a propiedades.
- La intención del código se vuelve más clara.
- Extraemos solo lo que necesitamos.
- Organizamos mejor la información.

5. Manejo de errores

El bloque **try...catch** es una estructura de JavaScript diseñada para **capturar y manejar errores durante la ejecución**, evitando que el programa se detenga por completo.

Estructura:

- **try**: contiene el código que podría generar un error.
- **catch**: captura el error si ocurre, permitiendo controlar la situación y continuar con la ejecución.

Ejemplo:

```
console.log("Inicio");
```

```
try {  
    const usuario = null;  
    console.log(usuario.nombre); // Esto generará un error  
} catch (error) {  
    console.error("Ocurrió un error:", error.message);  
}
```

```
console.log("Fin del programa"); // Sí se ejecuta gracias al manejo del error
```

¿Qué resolvemos?

- Evitamos que el programa se interrumpa.
- Informamos el error de manera clara y controlada.
- Mantenemos la ejecución estable.



- Permitimos tomar decisiones cuando algo falla (mostrar mensajes, usar valores alternos, guardar registros, etc.).
- Mejoramos la calidad y confiabilidad del software.

Actividades de aprendizaje:

Resuelve los siguientes enunciados

Ejercicio 1:

Crea una función llamada **registrarUsuarios** que reciba un primer argumento obligatorio con la información de un usuario principal (**nombre, edad**) y luego un número indeterminado de usuarios adicionales usando **parámetros rest**.

1. Usa **destructuración** para extraer nombre y edad del usuario principal.
2. La función debe mostrar un mensaje indicando cuántos usuarios adicionales se registraron.

Resultado esperado (ejemplo):

Usuario principal: Ana, Edad: 28 — Usuarios adicionales registrados: 3

Ejercicio 2:

Tienes un arreglo inicial:

```
const productos = ["teclado", "mouse", "pantalla"];
```

Crea una función **agregarProducto(lista, nuevoProducto)** que:

1. No modifique la lista original.
2. Devuelva una **nueva lista** usando el **operador spread**, agregando al final el nuevo producto.
3. Comprueba mostrando ambos arreglos que la inmutabilidad se mantiene.

Ejercicio 3:

Crea una función llamada **mostrarDireccion** que reciba un objeto con la siguiente estructura:



```
{ ciudad: "Bogotá", pais: "Colombia" }
```

La función debe:

1. Usar **destructuración** para obtener ciudad y pais.
2. Envolver el proceso dentro de un **try...catch**.
3. Si el objeto no contiene las propiedades necesarias, arrojar un error personalizado que indique:
"La información de la dirección no es válida".

Ejercicio 4:

Crea una función **actualizarNotas(estudiante, ...nuevasNotas)** que:

1. Reciba un objeto como:

```
const estudiante = {  
  nombre: "Laura",  
  notas: [4.0, 3.8]  
};
```
2. Use **spread** para crear un nuevo arreglo que combine las notas anteriores y las nuevas.
3. Devuelva un **nuevo objeto** (no mutar el original).
4. Verifica en consola que el objeto inicial no se modificó.

Ejercicio 5:

Crea un módulo de gestión básica de inventario con las siguientes características:

Función agregarItems(lista, ...items)

- Usa **rest** para recibir varios ítems.
- Devuelve una **nueva lista** usando **spread**, garantizando inmutabilidad.

Función obtenerInfo(item)

- Usa **destructuración** para extraer propiedades como id, nombre, precio.
- Si falta alguna propiedad, usa **try...catch** para capturar el error y mostrar un mensaje claro.



Requerimiento final

- Combina ambas funciones en una pequeña prueba donde:
 - Creas una lista inicial de ítems.
 - Agregas varios ítems a la vez.
 - Consultas la información de uno de ellos usando deestructuración y manejo de errores.

Materiales de formación:

Visual estudio code

Computador

Navegador

Estrategias didácticas activas:

- Clase magistral breve con mediación activa.
- Aprendizaje activo
- Aprendizaje significativo

Duración de la actividad 1 hora.

C. Actividades de apropiación del conocimiento.

En esta etapa del proceso de aprendizaje te invitamos a ir un poco más allá de la comprensión básica de los conceptos. Hasta este punto has trabajado ideas esenciales para escribir código más claro, seguro y mantenible: el uso de parámetros rest, el operador spread, la inmutabilidad, la deestructuración y el manejo de errores con try...catch.

Estos elementos son parte del estilo moderno de JavaScript y constituyen la base de muchas prácticas actuales del desarrollo, desde la creación de APIs hasta la construcción de interfaces interactivas. Dominar estas herramientas no solo te permitirá resolver



problemas de manera eficiente, sino también pensar como un desarrollador capaz de anticiparse a errores, separar responsabilidades y construir soluciones elegantes.

Los ejercicios que vienen a continuación buscan conectar estos temas con situaciones cada vez más cercanas a proyectos reales. La invitación es a reflexionar, explorar tus propias soluciones y comprobar cómo cada concepto encaja en la forma en que JavaScript maneja los datos y la lógica.

Actividades Prácticas individual

Resuelve los siguientes enunciados:

1. Gestión de pedidos con rest y destructuración

Crea una función **procesarPedido(pedido, ...extras)** donde *pedido* es un objeto con {cliente, producto, cantidad}.

- Usa **destructuración** para obtener sus propiedades.
- Usa **parámetros rest** para almacenar los extras del pedido.

La función debe retornar un objeto final con toda la información consolidada.

2. Control de inventario con inmutabilidad y spread

Partiendo del arreglo inicial:

```
const inventario = ["cámara", "trípode", "micrófono"];
```

Crea una función **agregarInventario(lista, nuevotem)** que devuelva una **nueva lista** sin modificar la original y que registre un mensaje indicando cuántos ítems tiene ahora el inventario.

3. Registro de actividades con manejo de errores

Escribe una función **registrarActividad(actividad)** que reciba un objeto que debe contener {nombre, fecha}.

- Usa un **try...catch** para validar que ambas propiedades existan.



- Si falta alguna, lanza un error indicando que la actividad no es válida.
- Si todo está correcto, muestra un mensaje confirmando el registro.

4. Mezcla de configuraciones con spread y destructuración

Simula un sistema donde existen opciones por defecto y opciones personalizadas:

```
const defaults = { tema: "claro", idioma: "es" };
```

Crea una función **configurarUsuario(defaults, personalizadas)** que:

- Use **spread** para combinar ambas configuraciones sin mutarlas.
- Use **destructuración** para obtener el idioma configurado.

Retorna el idioma resultante.

5. Actualización de perfiles con inmutabilidad + rest

Crea una función **actualizarPerfil(perfil, ...nuevosDatos)**.

Cada elemento de *nuevosDatos* será un objeto parcial como {edad: 22} o {ciudad: "Medellín"}.

- Combina todos los datos usando **inmutabilidad** y **spread**.

Retorna el nuevo perfil completo.

5. Análisis de ventas con destructuración profunda

Dado este arreglo:

```
const ventas = [  
  { producto: "teclado", detalles: { precio: 50, unidades: 3 } },  
  { producto: "mouse", detalles: { precio: 20, unidades: 5 } }  
];
```

Crea una función que emplee **destructuración anidada** para obtener precio y unidades del primer producto, y retorne el total vendido (precio × unidades).

7. Evaluación de datos con try...catch + spread

Escribe una función **evaluar(...valores)** que:



- Reciba varios números mediante **rest**.
- Use un **try...catch** para verificar que todos sean numéricos.
- Si encuentra un valor no numérico, captura el error y muestra un mensaje adecuado.
- Si todo es válido, crea una nueva lista (sin mutar la original) y retorna el promedio.

8. Fusión de colecciones y validación final

Crea una función **fusionarColecciones(lista1, lista2)** que:

- Use **spread** para fusionar las dos listas sin mutarlas.
- Valide dentro de un **try...catch** que ambas listas sean arreglos.
- Si no lo son, lanza un error personalizado.
- Retorna la colección final.

Materiales de formación:

- Planteamiento de los ejercicios
- Pc
- Navegador
- Visual estudio code

Estrategias didácticas activas:

- Aprendizaje activo:
- Resolución de problemas prácticos
- Trabajo autónomo guiado

Evidencias de aprendizaje:

- Código fuente con repositorio



Duración de la actividad 2 horas.

D. Actividades de transferencia del conocimiento.

Durante esta guía has recorrido un proceso que te ha permitido avanzar desde la comprensión inicial del problema, pasando por una exploración guiada de conceptos claves del JavaScript moderno —como los **parámetros rest**, el **operador spread**, la **inmutabilidad**, la **destructuración** y el **manejo de errores con try...catch**— hasta llegar a su aplicación contextualizada en ejercicios de complejidad creciente.

Ahora te encuentras en el **momento de transferencia**, una fase diseñada para que lleves tus nuevos conocimientos a situaciones donde deberás analizar, planear y resolver desafíos con un mayor nivel de autonomía. Aquí es donde conectas lo aprendido con problemas reales de programación, demostrando tu capacidad para aplicar técnicas de código limpio, prevenir errores, tomar decisiones y estructurar soluciones robustas.

Te invito a desarrollar los ejercicios con actitud reflexiva, curiosidad y disposición para experimentar. Cada actividad te permitirá comprobar cómo estas herramientas fortalecen tu pensamiento lógico y tu habilidad para construir soluciones claras y seguras en JavaScript. Este es el paso donde tu aprendizaje se consolida definitivamente.

Actividades de aprendizaje:

Ejercicios Prácticos

1. Sistema de registro académico

Requerimiento:

Crea una función **crearEstudiante(nombre, ...notas)** que reciba:

- Un nombre obligatorio.
- Un número variable de notas mediante **rest**.



La función debe:

1. Validar con **try...catch** que todas las notas sean números.
2. Usar **deestructuración** para separar la primera nota del resto.
3. Retornar un objeto inmutable con:
 - Nombre
 - Primera nota
 - Promedio del resto de notas
 - Total de notas registradas

Resultado esperado:

Un objeto final correctamente construido, sin mutar ningún arreglo externo, con cálculos correctos y con manejo adecuado de errores.

2. Fusión de catálogos digitales

Requerimiento:

Tienes dos catálogos:

```
const catalogoA = [  
  { id: 1, nombre: "Curso JavaScript", precio: 40 },  
  { id: 2, nombre: "Curso HTML", precio: 35 }  
];  
  
const catalogoB = [  
  { id: 3, nombre: "Curso CSS", precio: 30 }  
];
```

Crea una función **fusionarCatalogos(a, b)** que:



1. Use **try...catch** para validar que ambos parámetros sean arreglos.
2. Utilice **spread** para fusionarlos sin modificar los originales.
3. Retorne un nuevo catálogo ordenado por precio ascendente.

Resultado esperado:

Una nueva colección combinada, ordenada e inmutable. Si un parámetro no es un arreglo, debe mostrarse un error comprensible.

3. Procesamiento de compras

Requerimiento:

Implementa una función **procesarCompra(cliente, productos)** donde:

- *cliente* es un objeto con {nombre, correo}.
- *productos* es una lista de objetos con {nombre, precio}.

La función debe:

1. Validar con **try...catch** que el cliente tenga los dos datos y que los productos sean válidos.
2. Utilizar **spread** para crear un nuevo objeto con toda la información del cliente.
3. Usar **destructuración** para separar el primer producto comprado del resto.
4. Retornar un informe con:
 - Total de productos
 - Precio total
 - Primer producto adquirido

Resultado esperado:

Un informe completo y coherente, construido con técnicas de inmutabilidad y manejo seguro de errores.

4. Informe de estadísticas deportivas

Requerimiento:

Partiendo del arreglo:

```
const jugadores = [
```

```
{ nombre: "Ana", stats: { puntos: 20, asistencias: 5 } },
```



```
{ nombre: "Luis", stats: { puntos: 15, asistencias: 7 } }  
];
```

Crea una función **estadisticas(jugadores)** que:

1. Verifique mediante **try...catch** que la estructura de datos sea válida.
2. Emplee **destructuración profunda** para obtener los puntos del primer jugador.
3. Calcule la suma total de puntos del equipo utilizando técnicas inmutables.
4. Devuelva un objeto con:
 - Puntos del primer jugador
 - Puntos totales del equipo
 - Lista inmutable de jugadores procesados

Resultado esperado:

Un objeto estadístico confiable, con cálculos correctos y sin mutación de la lista original.

5. Motor de configuración avanzada

Requerimiento:

Dispones de estas configuraciones:

```
const baseConfig = { modo: "producción", lenguaje: "es", nivel: 1 };
```

```
const extraConfig = { nivel: 2, tema: "oscuro" };
```

Crea la función **configFinal(...configs)** que:

1. Reciba múltiples configuraciones usando **rest**.
2. Use **spread** para mezclarlas todas en un solo objeto final sin mutar ninguna.
3. Valide con **try...catch** que cada elemento recibido sea un objeto.
4. Retorne un objeto final con una propiedad adicional:
 - validacion: true si todo salió bien



- validacion: false si ocurrió un error

Resultado esperado:

Un objeto final consolidado, estable y construido con prácticas de programación segura, que refleje el uso combinado de todos los conceptos de la guía.

Para desarrollar los 5 ejercicios es requerido trabajar con la siguiente estructura:

- Archivo index.html para dar inicio al programa.
- Archivo app.js para poder indicar que ejercicio se desea ejecutar.
 - Se debe solicitar al usuario por medio de un menu manejado por un prompt que ejercicio desea solucionar.
 - Si el usuario no menciona el ejercicio indicado el sistema debe solicitar nuevamente que indique que ejercicio desea solucionar sin tener que recargar la página.
- Directorio de módulos para trabajar de forma ordenada los ejercicios
- Archivo barril para ordenar las importaciones y exportaciones

Materiales de formación:

- Editor de código (VS Code)
- Navegador web con herramientas de inspección
- Repositorio de Git

Estrategias didácticas activas:

- Resolución de problemas prácticos.
- Aprendizaje significativo en contextos cotidianos.
- Trabajo autónomo con retroalimentación del instructor.
- Uso de GitHub como portafolio de evidencias.

Evidencias de aprendizaje:



- La solución de los ejercicios en archivos separados
- Comentarios en el código explicando el uso de condicionales.
- Registro de commits claros en GitHub.
- Comentarios de los análisis requeridos

Duración de la actividad 3 horas.

4) ACTIVIDADES DE EVALUACIÓN

Tome como referencia la técnica e instrumentos de evaluación citados en la guía de Desarrollo Curricular

Fase del proyecto formativo	Actividad del proyecto formativo	Actividad de Aprendizaje	Evidencias de Aprendizaje	Criterios de Evaluación	Técnicas e Instrumentos de Evaluación
EJECUCIÓN	DESARROLLAR LA ESTRUCTURA DE DATOS, CONSTRUCCIÓN, CODIFICACIÓN E INTERFAZ DE USUARIO, PRUEBAS Y VALIDACIÓN DE LOS MÓDULOS DEL SISTEMA DE INFORMACIÓN	Identifica y describe, en un sistema de información dado, los datos de entrada, procesamiento de los datos e información generada, según necesidades del cliente.	Conocimiento y Producto	Solucionar problemas de lógica proposicional. Usar estructuras secuenciales en la construcción de algoritmos. Utilizar estructuras de control en la construcción de algoritmos. Emplear estructuras cíclicas en la construcción	Listas de chequeo



				de algoritmos. Construir algoritmos con arreglos.	
--	--	--	--	---	--

5) GLOSARIO DE TÉRMINOS

Parámetros Rest: Mecanismo que permite a una función recibir un número indefinido de argumentos agrupándolos en un solo arreglo. Facilita la flexibilidad en funciones que necesitan manejar múltiples valores sin especificarlos uno a uno.

Operador Spread: Operador que permite expandir elementos de arreglos u objetos dentro de nuevas estructuras. Se usa para clonar, combinar o copiar datos sin modificar los originales, promoviendo buenas prácticas de inmutabilidad.

Inmutabilidad: Principio de programación que consiste en **no modificar directamente** una estructura de datos original, sino generar una nueva versión a partir de ella. Esto evita efectos secundarios, errores silenciosos y comportamientos inesperados en el código.

Mutabilidad: Característica de ciertas estructuras (como los objetos y arreglos en JavaScript) que permiten ser modificadas directamente en memoria. Aunque es válida, su uso descontrolado puede generar errores difíciles de rastrear.

Desestructuración (Destructuring): Sintaxis de JavaScript que permite extraer valores de arreglos u objetos y asignarlos fácilmente a variables individuales. Facilita la lectura del código y hace más clara la manipulación de datos complejos.

Desestructuración de Arreglos: Técnica mediante la cual se extraen valores de un arreglo según su posición.

Desestructuración de Objetos: Técnica que permite extraer propiedades de un objeto según su nombre.



Desestructuración Profunda: Uso de la desestructuración en niveles internos de un objeto complejo.

try...catch: Estructura de control que permite manejar errores en tiempo de ejecución.

- **try:** contiene el código que podría generar un error.
- **catch:** captura y gestiona el error, evitando que el programa se detenga abruptamente.

Error Personalizado: Error creado manualmente mediante throw para indicar que ocurrió una situación inesperada o inválida en la lógica del programa. Permite al desarrollador ofrecer mensajes claros y específicos.

Función: Bloque de código reutilizable que encapsula instrucciones para realizar una tarea específica. Puede recibir parámetros y devolver resultados.

Arreglo (Array): Estructura de datos ordenada que almacena múltiples valores en una sola variable. Cada valor es accesible mediante un índice numérico que comienza en 0.

Objeto Literal: Estructura clave–valor que permite organizar información en forma de propiedades.

Clonación Superficial: Copia de un objeto o arreglo donde solo el primer nivel es duplicado; niveles internos siguen apuntando a la misma referencia en memoria.

Flujo de datos: Forma en que la información se mueve y transforma dentro del programa. Comprenderlo ayuda a escribir código predecible y organizado.

Validación de datos: Proceso de revisar que los valores recibidos cumplan con las características esperadas (tipo, formato, contenido). Es esencial para evitar errores en la ejecución.

Argumentos: Valores que se envían a una función cuando es invocada. Pueden ser uno, varios o incluso ninguno según la necesidad.

Estado del programa: Representación del conjunto de valores almacenados en variables, objetos y estructuras en un momento determinado. La inmutabilidad ayuda a mantener ese estado controlado.



6) REFERENTES BIBLIOGRÁFICOS

- Deitel, P., & Deitel, H. (2019). *Cómo programar en Java* (11.^a ed.). Pearson Educación.
- Joyanes Aguilar, L. (2017). *Fundamentos de programación: Algoritmos, estructuras de datos y objetos*. McGraw-Hill.
- Malvino, A. P., & Brown, J. A. (2018). *Principios de programación estructurada*. McGraw-Hill.
- Pressman, R. (2021). *Ingeniería de software: Un enfoque práctico* (9.^a ed.). McGraw-Hill.
- Forbellone, A. L., & Eberspächer, H. F. (2000). *Lógica de programación: La construcción de algoritmos y estructuras de datos*. Prentice Hall.
- Freire, P. (2005). *Pedagogía del oprimido*. Siglo XXI Editores.
- SENA (2022). *Guía metodológica para la elaboración de guías de aprendizaje*. Servicio Nacional de Aprendizaje – SENA.
- W3Schools. (s.f.). *JavaScript Loops*. Recuperado de: https://www.w3schools.com/js/js_loop_for.asp
- GeeksforGeeks. (s.f.). *Loops in Programming Languages*. Recuperado de: <https://www.geeksforgeeks.org/loops-in-programming-languages>

7) CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	John Freddy Becerra Castellanos	Instructor Contratista	CIMI	11/20/2025

8) CONTROL DE CAMBIOS (diligenciar únicamente si realiza ajustes a la guía)

	Nombre	Cargo	Dependencia	Fecha	Razón del Cambio
--	--------	-------	-------------	-------	---------------------



Autor (es)					
-----------------------	--	--	--	--	--